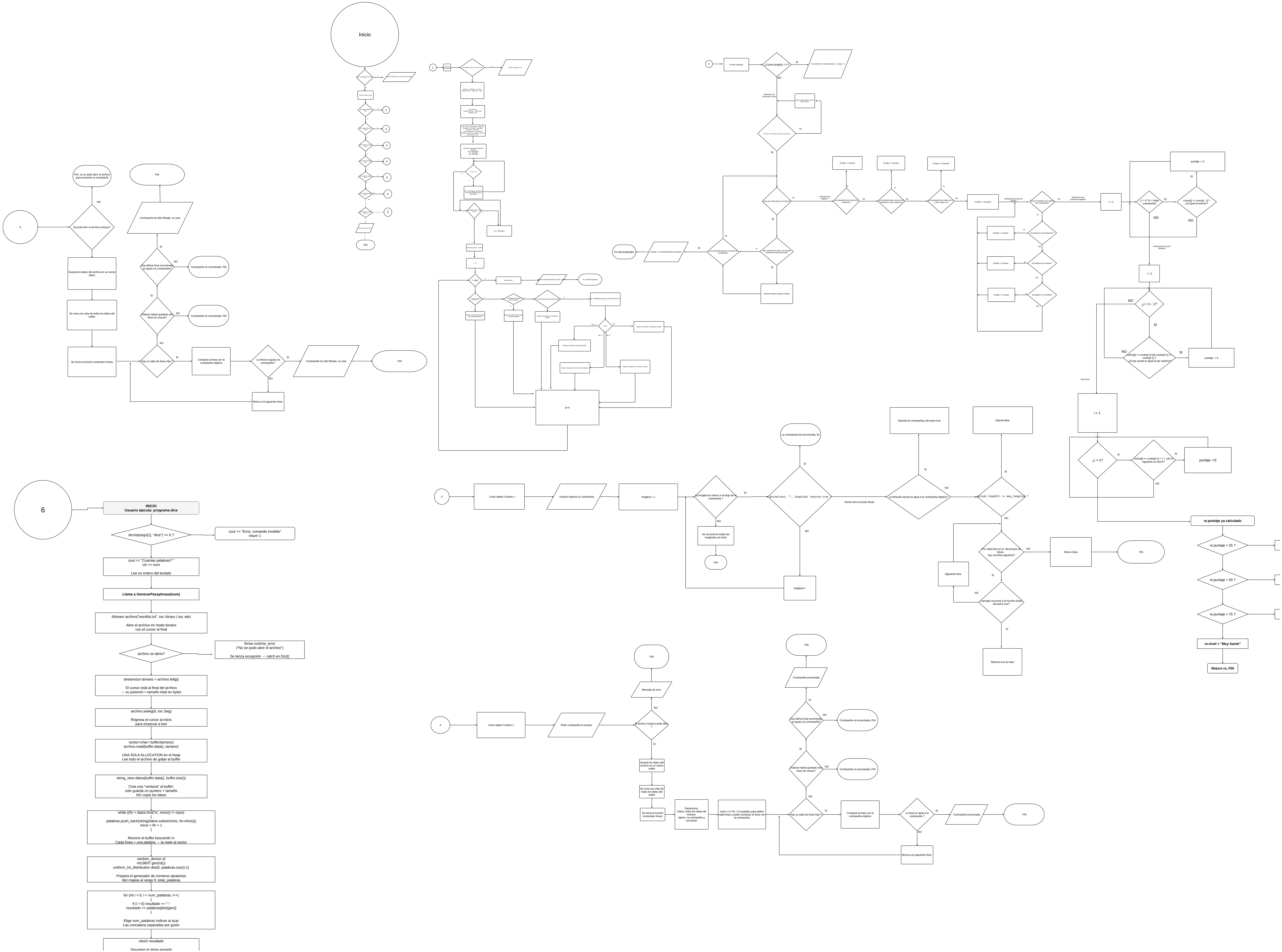
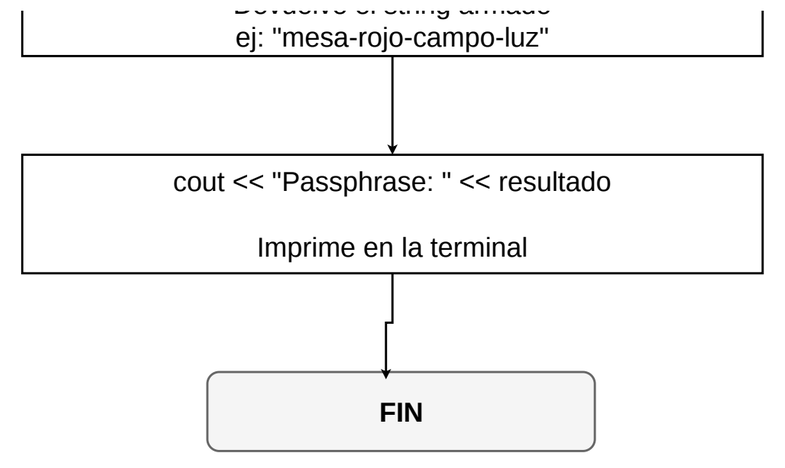

Inicio
¿num de argumentos menor a 2?
SI
Mostrar mensaje de error y opciones de comandos
NO
Evaluar comando escrito
¿El comando es igual a "gen"?
SI
1
NO
¿El comando es igual a "check"?
SI
2
NO
¿El comando es igual a "brute"?
SI
3
NO
¿El comando es igual a "dict" ?
SI
4
NO
¿El comando es igual a "pwn" ?
SI
5
NO
¿El comando es igual a "dice" ?
Si
6
No
¿El comando es igual a "ritmo" ?
7
Mostrar mensaje de error
FIN
1
Funcion Generar(int)
¿La longitud es mayor a 6 y menor a 64?
No
Mostrar mensaje de error
Definir cadenas: letras, mayusculas, simbolos, nums
Inicializar random_device, seed_seq, mt19937 gen
Crear distribuciones: distLet, distMay, distSim, distNum, distPos, distTipo que escogerán un caracter aleatorio de las cadenas letras, mayusculas etc.
Generamos numeros de posiciones aleatorias uno = distPos(gen) dos = distPos(gen) tres = disPos(gen)
¿Dos igual a uno?
NO
Si
dos = distPos(gen), generamos otro numero aleatorio para la posicion dos
¿tres igual a dos o igual a uno?
SI
NO
tres = distpos(gen)
Crear string vacío: "contra"
i = 0
i < largo?
NO
Salir del bucle
Mostrar contraseña generada al usuario: "contra"
Fin. Contraseña generada
i = uno (Numero generado aleatoriamente)
NO
SI
Asigna en la posición i del string una mayúscula aleatoria
i = dos(Numero generado aleatoriamente)
NO
SI
Asigna en la posición i del string un símbolo aleatorio
i = tres(Numero generado aleatoriamente)
NO
Si
Asigna en la posición i de la string un número
tipo = distipo(gen) genera un numero aleatorio del 1-4
¿Tipo?
tipo = 0
Otro
tipo = 1
tipo = 2
Asigna en la posición i del string una letra
Asigna en la posición i del string un símbolo
Asigna en la posición i del string una mayúscula
Asigna en la posición i del string un numero
i++
2
Función analizar()
¿Contra.length() = 0 ?
SI
"No puedes tener contraseña vacia" , puntaje = 0
NO
Verificación en diccionario común
Avisar al programador de que no jala su archivo
¿El archivo "10k-common-passwords" se pudo abrir?
NO
Si
¿Hay una nueva linea en el archivo?
No
Evaluación de longitud ->
Si
Hay una linea no vacía y un caracter invisible al final de esa linea?
NO
Si
Elimina el último caracter invisible
¿La contraseña aparece en la lista de contraseñas?
SI
NO
Puntaje = 0, contraseña fácil de crackear
Fin del analizador
¿La contraseña tiene menos de 6 carácteres?
Si
Puntaje += 0 puntos
NO
La contraseña tiene menos de 10 carácteres y más o igual a 6?
Si
Puntaje += 5 puntos
NO
La contraseña tiene menos de 12 y más o igual a 10
SI
Puntaje += 10 puntos
NO
Puntaje += 25 puntos
Bonificación por tipo de carácter
Hay otro caracter en la cadena de la contraseña?
NO
SI
El caracter es una mayúscula ?
SI
Puntaje += 2 puntos
NO
El caracter es un número ?
SI
Puntaje += 3 puntos
NO
El carácter es un símbolo?
SI
Puntaje += 4 puntos
NO
Penalización por carácteres repetidos
i = 1
¿i < n? (n = largo contraseña)
SI
contra[i] == contra[i - 1] ? ¿Es igual al anterior?
SI
puntaje -= 5
NO
NO
Penalización por pares repetidos
i = 2
¿i < n - 1?
NO
SI
contra[i] == contra[i-2] && contra[i+1] == contra[i-1] ? ¿El par actual es igual al par anterior?
NO
SI
puntaje -= 5
Secuencias
i = 1
¿i < n?
SI
contra[i] == contra[i-1] + 1 ? ¿Es el siguiente en ASCII?
SI
puntaje -=5
NO
re.puntaje ya calculado
re.puntaje < 25 ?
re.puntaje < 50 ?
re.puntaje < 75 ?
re.nivel = "Muy fuerte"
Return re, FIN
5
Se pudo leer el archivo rockyou?
NO
FIN. no se pudo abrir el archivo para encontrar la contraseña
Guarda los datos del archivo en un vector datos
Se crea una vista de todos los datos del buffer
Se inicia la función comprobar líneas
Hay un salto de linea más ?
SI
Compara la linea con la contraseña objetivo
La linea es igual a la contraseña ?
SI
Contraseña ha sido filtrada, no usar
FIN
NO
Brinca a la siguiente línea
NO
Todavía habrá quedado otra linea sin checar?
NO
Contraseña no encontrada. FIN
SI
Esa última línea encontrada es igual a la contraseña?
NO
Contraseña no encontrada. FIN
SI
Contraseña ha sido filtrada, no usar
FIN
3
Crear objeto Cracker c
Usuario ingresa su contraseña
longitud = 1
La longitud es menor a el largo de la contraseña ?
NO
Se recorrieron todas las longitudes sin écito
FIN
SI
¿Brute(cont, "", longitud) retorna true ?
SI
La contraseña fue encontrada, fin
NO
longitud++
Dentro de la función Brute
La iteración actual es igual a la contraseña objetivo?
SI
Muestra la contraseñay devuelve true
NO
¿actual.length() >= max_longitud ?
SI
retorna false
NO
Por cada letra en el diccionario de letras , hay una letra siguiente?
NO
Return false
FIN
SI
Llamada recursiva a la función brute, devuelve true?
NO
Siguiente letra
SI
Retorna true al main
4
Crear objeto Cracker c
Pedir contraseña al usuario
El archivo rockyou pudo abrir ?
NO
Mensaje de error
FIN
SI
Guarda los datos del archivo en un vector buffer
Se crea una vista de todos los datos del buffer
Se inicia la función comprobar líneas
Parametros: Datos: todos los datos de rockyou bjetivo: la contraseña a encontrar
inicio = 0. Fin = 0,variables para definir cada línea y poder comparar el texto con la contraseña
Hay un salto de linea más ?
SI
Compara la linea con la contraseña objetivo
La linea es igual a la contraseña ?
SI
Contraseña encontrada
FIN
NO
Brinca a la siguiente línea
NO
Todavía habrá quedado otra linea sin checar?
NO
Contraseña no encontrada. FIN
SI
Esa última línea encontrada es igual a la contraseña?
NO
Contraseña no encontrada. FIN
SI
Contraseña encontrada
FIN
6
INICIO
Usuario ejecuta: programa dice
strcmp(argv[1], "dice") == 0 ?
cout << "Error, comando invalido"
return 1
cout << "Cuantas palabras? "
cin >> num
Lee un entero del teclado
Llama a GenerarPassphrase(num)
ifstream archivo("wordlist.txt", ios::binary | ios::ate)
Abre el archivo en modo binario con el cursor al final
archivo se abrio?
throw runtime_error ("No se pudo abrir el archivo")
Se lanza excepción → catch en Dict()
streamsize tamano = archivo.tellg()
El cursor está al final del archivo → su posición = tamaño total en bytes
archivo.seekg(0, ios::beg)
Regresa el cursor al inicio para empezar a leer
vector<char> buffer(tamano)
archivo.read(buffer.data(), tamano)
UNA SOLA ALLOCATION en el heap
Lee todo el archivo de golpe al buffer
string_view datos(buffer.data(), buffer.size())
Crea una "ventana" al buffer: solo guarda un puntero + tamaño NO copia los datos
while ((fin = datos.find('\n', inicio)) != npos)
{
palabras.push_back(string(datos.substr(inicio, fin-inicio)))
inicio = fin + 1
}
Recorre el buffer buscando \n
Cada línea = una palabra → la mete al vector
random_device rd
mt19937 gen(rd())
uniform_int_distribution dist(0, palabras.size()-1)
Prepara el generador de números aleatorios
dist mapea al rango 0..total_palabras
for (int i = 0; i < num_palabras; i++)
{
if (i > 0) resultado += "-"
resultado += palabras[dist(gen)]
}
Elige num_palabras índices al azar
Las concatena separadas por guión
return resultado
Devuelve el string armado

re.nivel = "Debil"
re.nivel = "Regular"
re.nivel = "Fuerte"


Devuelve el string armado
ej: "mesa-rojo-campo-luz"
cout << "Passphrase: " << resultado
Imprime en la terminal
FIN